

Toward a Complete Digital Twin of Hub Sort Operations: An Idealized Multi-Layer Mathematical Framework Integrating External Logistics, Physical Infrastructure, and Human Factors

Rafael Almeida (Employee 6068314) — UPS Chelmsford Hub

May 2026

Version: 2.0 — Idealized Digital Twin Framework (Research Vision Branch)

Status: Speculative theoretical extension — not yet production-validated

Relation to v1.4: v1.4 is a **restricted case** of this framework under the assumption that all external, infrastructure, and environmental layers are unobserved.

WARNING: This branch does not replace v1.4 or the incremental v1.5 path. It defines a long-horizon research target.

Abstract

The v1.4 framework establishes a mathematically rigorous three-timescale model of the UPS Chelmsford package sort, validated against 633 historical sorts. That framework treats the sort as a **closed system** — it ingests data only from iGate scanning and SOR staffing records, with no visibility into the external logistics network that determines the volume, timing, and composition of incoming freight.

This paper formulates **v2.0**, an idealized extension that opens the sort to its full causal environment. We introduce five additional observational layers: (1) an **external arrival layer** modeling feeder truck and delivery driver GPS positions as a marked point process with Bayesian ETA updating; (2) an **environmental layer** coupling weather state (temperature, precipitation, visibility, road conditions) to traffic delay amplification and accident probability via a coupled Markov-modulated process; (3) a **physical infrastructure layer** formalizing the conveyor belt system as a time-inhomogeneous M/G/1 queue with controlled service rate (belt speed), a jam probability model under high induction density, and chute accumulation dynamics; (4) a **package composition layer** connecting SLIC-code distributions and flowable/non-flowable fractions to belt capacity consumption and zone load imbalance; and (5) a **human factors layer** encoding fatigue degradation curves, experience heterogeneity, and break schedule discontinuities. These layers are unified in a **high-dimensional state space model** tracked by an Extended Kalman Filter, enabling predictive simulation of any observable quantity — PPH, belt congestion, staffing needs, or sort completion time — from pre-sort sensor data alone.

v2.0 is explicitly designated a **research vision branch**. It identifies what data must exist, what sensors must be deployed, and what mathematical infrastructure must be in place before this model can be realized in production. v1.4 remains the authoritative operational baseline; v1.5 will extend

it incrementally. v2.0 is the north star.

Keywords: digital twin; package sorting; stochastic arrival process; M/G/1 queue; Extended Kalman Filter; weather-traffic coupling; fatigue model; induction rate control; GPS telematics; closed-loop optimization.

1. Introduction

1.1 The Open-System Problem

Every quantity the v1.4 framework predicts — PPH trajectories, staffing needs, sort quality scores — is determined not only by what happens inside the building but by what is approaching it. A feeder delayed on I-495 by a winter storm delivers its 1,800 pieces twenty minutes late; the inbound ramp compresses, the induction rate spikes, jam probability increases, and sorters who would have been in steady-state are abruptly handling peak-phase density. The v1.4 model sees none of this until the packages are on the belt. It infers external dynamics from their internal consequences, after the fact.

This is not a failure of v1.4. It is an accurate description of what data is currently available. The Tracker and MasterTrendAnalysis operate on iGate and SOR exports — both are post-hoc records of what happened, not pre-hoc predictions of what is incoming. Under current data constraints, v1.4 is the correct model.

v2.0 asks: *if all relevant data existed and were accessible in real time, what would the complete mathematical model look like?*

1.2 Five New Causal Layers

The v1.4 framework is a **single-layer model**: it observes the interior of the sort (scans, hours, PPH) and predicts interior quantities. v2.0 adds five layers that feed causally into sort dynamics before they appear in iGate data:

Layer	Data Sources	Causal Role
External Arrival	Feeder GPS, delivery driver GPS, ORION manifests	Determines volume, timing, and phasing of package flow
Environmental	NOAA weather API, Waze/HERE traffic, DOT incident feeds	Modulates feeder ETAs, road delay, driver safety margins
Physical Infrastructure	Belt speed sensors, induction rate counters, chute load cells	Sets throughput ceiling, jam probability, sort capacity
Package Composition	SLIC manifests, package dimensions (vision system), weight sensors	Determines belt capacity consumption, zone load distribution
Human Factors	Biometric wearables (opt-in), time-on-task, break records	Modulates effective PPH as function of fatigue and experience

1.3 v2.0 as Idealized Restriction of a General Model

Formally, the v1.4 framework is obtained from v2.0 by conditioning on the unobservability of all five new layers. Every theorem in v1.4 remains valid in v2.0 — they are simply theorems about the

marginal dynamics of the iGate/SOR subsystem when external state is integrated out. v2.0 does not revise v1.4; it embeds it.

1.4 Organization

Section 2 defines the complete system architecture. Sections 3–7 develop each new layer mathematically. Section 8 unifies them in a high-dimensional state space model with Extended Kalman Filter. Section 9 derives the integrated optimization framework. Section 10 presents the digital twin architecture. Section 11 formalizes the connection to v1.4 as a restricted case. Section 12 enumerates the data requirements that must be met before v2.0 can be instantiated. Sections 13–14 address limitations and future research directions.

2. System Architecture

2.1 The Hub as a Controlled Queueing Network

The UPS Chelmsford sort can be modeled as a **controlled queueing network** $\mathcal{H} = (\mathcal{N}, \mathcal{E}, \mathcal{C}, \mathcal{U})$ where:

- $\mathcal{N}$ is the set of nodes: induction stations, belt segments, sort chutes, unload doors, load doors
- $\mathcal{E}$ is the set of directed edges (package flow paths)
- $\mathcal{C} : \mathcal{E} \rightarrow \mathbb{R}_+$ assigns capacity (packages/min) to each edge
- $\mathcal{U}$ is the control space: belt speed settings, induction rate targets, staffing assignments, door open/close states

Packages arrive at induction nodes according to an externally-driven **arrival process** $\Lambda(t)$ (Section 3), traverse the belt network subject to capacity constraints $\mathcal{C}$ and jam dynamics (Section 5), and exit at sort chutes with a routing distribution determined by the sorter channel model $\mathbf{M}$ (v1.4, §4).

2.2 Multi-Layer State Vector

The complete state of the system at time $t \in [0, 240]$ (minutes from sort start) is:

$$\mathbf{X}(t) = [\underbrace{\mathbf{x}_{\text{ext}}(t)}_{\text{External}}, \underbrace{\mathbf{x}_{\text{env}}(t)}_{\text{Environmental}}, \underbrace{\mathbf{x}_{\text{infra}}(t)}_{\text{Infrastructure}}, \underbrace{\mathbf{x}_{\text{pkg}}(t)}_{\text{Package}}, \underbrace{\mathbf{x}_{\text{human}}(t)}_{\text{Human}}, \underbrace{\mathbf{x}_{\text{ops}}(t)}_{\text{v1.4 Ops}}]$$

The v1.4 state vector $\mathbf{x}_{\text{ops}}(t)$ — containing PPH, staffing counts, fidelity score, SQS components — is the **observable projection** of the full state $\mathbf{X}(t)$ onto the iGate/SOR subspace. All upstream layers are latent in the v1.4 model but explicit in v2.0.

3. External Arrival Layer

3.1 The Freight Arrival Process

Definition 3.1 (Marked Point Process). The inbound freight arrival process is a marked point process $\Phi = \{(\tau_i, \mathbf{m}_i)\}_{i=1}^N$ on $[t_0 - \Delta, T]$ (beginning Δ minutes before sort start $t_0 = 0$, ending at sort close $T = 240$), where:

- $\tau_i \in \mathbb{R}$ is the arrival time of vehicle i at the hub dock
- $\mathbf{m}_i = (V_i, \mathbf{s}_i, \text{type}_i)$ is the mark: total piece count V_i , SLIC vector $\mathbf{s}_i \in \mathbb{R}^{|\mathcal{Z}|}$ (zone load distribution), and vehicle type $\text{type}_i \in \{\text{feeder}, \text{P-car}, \text{air}\}$

The arrival intensity function $\lambda(t)$ is non-homogeneous, with characteristic feeder pulses at approximately $t \in \{-60, 0, 30, 90\}$ minutes relative to sort start (pre-loaded, first wave, second wave, peak feeder wave) and a distributed P-car tail through $t \in [30, 180]$ as delivery drivers return from route.

Definition 3.2 (GPS-Based ETA Estimator). For each vehicle i with GPS position $\mathbf{p}_i(t) \in \mathbb{R}^2$, velocity $v_i(t)$, and route $\mathcal{R}_i$ (polyline to hub), the ETA estimate at time t is:

$$\hat{\tau}_i(t) = t + \int_{\mathbf{p}_i(t)}^{\mathbf{p}_{\text{hub}}} \frac{d\ell}{v_i(\ell, t, W(t))} + \epsilon_i(t)$$

where $v_i(\ell, t, W(t))$ is speed at road segment ℓ conditioned on weather state $W(t)$ (Section 4), and $\epsilon_i(t) \sim \mathcal{N}(0, \sigma_{\tau,i}^2(t))$ captures GPS uncertainty and unmodeled delays. ETA uncertainty $\sigma_{\tau,i}^2(t)$ decreases as i approaches: $\sigma_{\tau,i}^2(t) = \sigma_0^2 \cdot d_i(t)/d_0$ where $d_i(t)$ is remaining distance.

Theorem 3.1 (Volume Arrival Forecast). The expected cumulative volume arriving by time t is:

$$\mathbb{E}[V_{\text{total}}(t)] = \sum_{i=1}^N V_i \cdot \Phi\left(\frac{t - \hat{\tau}_i(t_0)}{\sigma_{\tau,i}(t_0)}\right)$$

where $\Phi(\cdot)$ is the standard normal CDF. This provides a probabilistic inbound forecast from the pre-sort GPS snapshot, with confidence bands narrowing as arrival time approaches.

Proof. Volume from vehicle i arrives at hub at time τ_i . Given $\tau_i \sim \mathcal{N}(\hat{\tau}_i, \sigma_{\tau,i}^2)$, $P(\tau_i \leq t) = \Phi((t - \hat{\tau}_i)/\sigma_{\tau,i})$. Total expected volume is by linearity of expectation over independent vehicle arrivals. $\square$

3.2 Feeder vs. Package Car Arrival Dynamics

Definition 3.3 (Feeder Pulse Model). A feeder arrival at time τ_i generates a **volume pulse** at the induction nodes:

$$\lambda_{\text{induct}}(t|\tau_i, V_i) = V_i \cdot \frac{r_{\text{unload}}}{\Delta t_{\text{unload}}} \cdot \mathbf{1}[\tau_i \leq t \leq \tau_i + \Delta t_{\text{unload}}]$$

where r_{unload} is the unload rate (pieces/min, dependent on staffing at that door), and $\Delta t_{\text{unload}} = V_i/r_{\text{unload}}$ is the unload duration. Multiple simultaneous feeders create **superimposed pulses** — a key driver of induction rate spikes and downstream belt congestion.

Definition 3.4 (P-Car Return Distribution). Delivery driver (P-car) return times follow a distribution shaped by route density and completion rate:

$$f_{\text{Pcar}}(\tau) = \text{LogNormal}(\mu_{\text{route}}, \sigma_{\text{route}}^2) \cdot \mathbf{1}[\tau \in [30, 180]]$$

with parameters estimated from historical ORION route completion data. Each P-car contributes a small volume ($\bar{V}_{\text{Pcar}} \approx 5\text{--}30$ pieces: undeliverables, pickups, returns), but the aggregate over 80–120 simultaneous drivers creates a sustained background arrival rate.

3.3 Volume Forecast Integration with Kalman Filter

The GPS-based volume forecast (Theorem 3.1) is fed as an **exogenous input** to the v1.4 Kalman filter prediction step:

$$\text{PPH}_{\text{predicted}}(t + \delta) = f\left(\text{PPH}(t), \left.\frac{d\mathbb{E}[V_{\text{total}}]}{dt}\right|_t, N(t), \phi(t)\right)$$

where $\phi(t) \in \{\text{Ramp, Steady, Wind-Down, Post}\}$ is the current phase and $d\mathbb{E}[V_{\text{total}}]/dt$ is the expected induction rate implied by approaching vehicles.

4. Environmental Layer

4.1 Weather State Markov Chain

Definition 4.1 (Weather State Space). The weather state $W(t)$ takes values in $\mathcal{W} = \{\text{Clear, Rain, HeavyRain, Snow, Ice, Fog}\}$ and evolves as a continuous-time Markov chain with generator matrix $\mathbf{Q}_W$ estimated from NOAA historical data for the Chelmsford, MA area by month and season.

Definition 4.2 (Weather-Speed Coupling). Road segment speed under weather state w is:

$$v(\ell, w) = v_{\text{free}}(\ell) \cdot \alpha_w \cdot \beta(\ell, w)$$

where $v_{\text{free}}(\ell)$ is free-flow speed, $\alpha_w \in (0, 1]$ is a weather attenuation factor:

State w	α_w (highway)	α_w (local)
Clear	1.00	1.00
Rain	0.85	0.80
HeavyRain	0.70	0.65
Snow	0.55	0.45
Ice	0.35	0.25
Fog	0.75	0.70

and $\beta(\ell, w)$ is a road-type correction (surface treatment, grade, curvature) estimated from DOT road geometry data.

4.2 Traffic Delay Amplification

Definition 4.3 (Traffic Density State). Traffic density on route $\mathcal{R}_i$ at time t is modeled via the Bureau of Public Roads (BPR) function:

$$v_{\text{traffic}}(\ell, t) = \frac{v_{\text{free}}(\ell)}{1 + 0.15 \left(\frac{q(\ell, t)}{c(\ell)} \right)^4}$$

where $q(\ell, t)$ is observed traffic flow (vehicles/hr) from HERE or Waze API, and $c(\ell)$ is road capacity (vehicles/hr/lane $\times$ lanes). Weather and traffic effects compound multiplicatively:

$$v_{\text{effective}}(\ell, t, w) = v_{\text{traffic}}(\ell, t) \cdot \alpha_w \cdot \beta(\ell, w)$$

Definition 4.4 (Accident Probability Model). The probability of a traffic incident on route $\mathcal{R}_i$ during the sort window is:

$$P(\text{accident on } \mathcal{R}_i) = 1 - \exp \left(- \int_{\mathcal{R}_i} \lambda_{\text{acc}}(\ell, w, q) d\ell \right)$$

where $\lambda_{\text{acc}}(\ell, w, q)$ is the local accident rate per unit length, modeled as:

$$\lambda_{\text{acc}}(\ell, w, q) = \lambda_0(\ell) \cdot \gamma_w \cdot \left(\frac{q(\ell)}{c(\ell)} \right)^\delta$$

with $\lambda_0(\ell)$ from historical MASSDOT incident data, γ_w a weather multiplier (Ice: 4.2 $\times$, Snow: 2.8 $\times$, HeavyRain: 1.6 $\times$, Rain: 1.2 $\times$, Clear: 1.0 $\times$), and $\delta \approx 1.4$ from empirical accident-density correlation studies (Lord & Mannering, 2010).

Theorem 4.1 (Delay Cascade). An accident at road segment ℓ^* with clearance time T_{clear} induces a delay cascade on all vehicles routed through ℓ^* with expected arrival delay:

$$\mathbb{E}[\Delta\tau_i | \text{accident at } \ell^*, T_{\text{clear}}] = T_{\text{clear}} + \frac{L_{\text{queue}}(t^*)}{v_{\text{discharge}}}$$

where $L_{\text{queue}}(t^*)$ is the queue length at accident clearance and $v_{\text{discharge}}$ is the discharge rate. This cascade propagates through the GPS-ETA estimator (Definition 3.2), updating $\hat{\tau}_i$ and triggering re-optimization of the staffing plan (Section 9.2).

4.3 Temperature Effects on Operations

Beyond traffic, weather directly affects sort operations:

Definition 4.5 (Heat-Fatigue Coupling). Sorter effective PPH under high ambient temperature T_{amb} ($^{\circ}\text{F}$) degrades according to:

$$\text{PPH}_{\text{eff}}(t, T_{\text{amb}}) = \text{PPH}_{\text{nominal}}(t) \cdot (1 - \kappa_T \cdot \max(0, T_{\text{amb}} - T_{\text{threshold}}))^+$$

with threshold $T_{\text{threshold}} \approx 80^{\circ}\text{F}$ (inside the building, affected by ambient outdoor temperature and building HVAC), and κ_T estimated from productivity studies in warm-environment manual labor (Seppänen et al., 2006: $\kappa_T \approx 0.02/^{\circ}\text{F}$ above threshold for sustained physical work). Cold temperatures ($T_{\text{amb}} < 50^{\circ}\text{F}$) produce slower initial ramp but may preserve late-sort PPH.

Definition 4.6 (Ice/Snow Loading Effect). Packages arriving via feeders during heavy precipitation carry surface moisture, which increases conveyor belt friction variability and can elevate jam probability (Section 5.3) by a factor $\eta_{\text{ice}} \in [1.1, 1.4]$ during Ice/Snow weather states.

5. Physical Infrastructure Layer

5.1 The Conveyor Belt as a Queue

Definition 5.1 (Belt Queue Model). Each belt segment $b \in \mathcal{B}$ is modeled as a time-inhomogeneous M/G/1 queue with:

- **Arrival rate:** $\lambda_b(t)$ — packages per minute arriving at segment b from upstream
- **Service rate:** $\mu_b(s_b) = s_b \cdot \rho_b$ — packages per minute processed, where s_b is belt speed (m/min) and ρ_b is packages per meter of belt (dependent on package size distribution)
- **Traffic intensity:** $\rho_b(t) = \lambda_b(t)/\mu_b(s_b)$

For stable belt flow, $\rho_b(t) < 1$ for all b and all t . The M/G/1 Pollaczek-Khinchine formula gives expected queue length:

$$\mathbb{E}[L_b] = \rho_b + \frac{\rho_b^2 + \lambda_b^2 \text{Var}[S_b]}{2(1 - \rho_b)}$$

where S_b is the service time distribution (dependent on package size variance — see §6.1).

Definition 5.2 (Belt Speed Control). Belt speed $s_b(t) \in [s_{\min}, s_{\max}]$ is a **control variable**. Increasing s_b raises μ_b and reduces congestion, but:

1. Increases package impact velocity at chute deflectors, elevating missort and damage risk
2. Reduces sorter reaction time per package, reducing effective scan accuracy (sorter DMC model)
3. Exceeds safe operating parameters above $s_{\max}$

The belt speed control policy (Section 9.3) optimizes throughput subject to these constraints.

5.2 Induction Rate Dynamics

Definition 5.3 (Induction Rate). The induction rate $\lambda_{\text{induct}}(t)$ (packages/min entering the system) is determined by unload staffing, feeder availability, and physical induction station capacity:

$$\lambda_{\text{induct}}(t) = \min(\lambda_{\text{raw}}(t), C_{\text{induct}}, \mu_{\text{main}}(s_{\text{main}}))$$

where $\lambda_{\text{raw}}(t) = \sum_{i: \tau_i \leq t} r_{\text{unload}, i}(t)$ is the raw arrival rate from active unload doors, C_{induct} is the physical induction station capacity (a fixed infrastructure constant), and $\mu_{\text{main}}(s_{\text{main}})$ is the main belt throughput capacity. The minimum enforces the bottleneck constraint.

Theorem 5.1 (Induction Ceiling). When $\lambda_{\text{raw}}(t) > \min(C_{\text{induct}}, \mu_{\text{main}})$, packages accumulate at the induction buffer at rate:

$$\frac{dB_{\text{induct}}}{dt} = \lambda_{\text{raw}}(t) - \min(C_{\text{induct}}, \mu_{\text{main}})$$

The buffer B_{induct} clears only when $\lambda_{\text{raw}}(t)$ drops below capacity. Buffer overflow forces door closure and package staging on the dock floor — a directly observable operational disruption. The total sort duration extends by $\Delta T_{\text{buffer}} = B_{\text{peak}}/(\mu_{\text{main}} - \bar{\lambda}_{\text{tail}})$ where $\bar{\lambda}_{\text{tail}}$ is average arrival rate during wind-down.

5.3 Jam Probability Model

A belt jam occurs when packages accumulate in a segment faster than the segment can process them, creating a physical blockage. Jams have two primary drivers: (a) belt density exceeding safe separation, and (b) irregular packages (oversized, wet, damaged) bridging adjacent packages.

Definition 5.4 (Jam Hazard Function). The instantaneous jam hazard rate at segment b is:

$$h_b(t) = h_0 \cdot \exp(\alpha_\rho(\rho_b(t) - \rho^*) + \alpha_{\text{irr}}f_{\text{irr}}(t) + \alpha_{\text{ice}}\eta_{\text{ice}}(t))$$

where h_0 is the baseline jam rate (calibrated per-belt from historical maintenance logs), $\rho^* \approx 0.75$ is the critical density above which jam risk increases sharply, $f_{\text{irr}}(t)$ is the fraction of irregular packages in the current induction stream, and η_{ice} is the weather loading factor (Definition 4.6).

Definition 5.5 (Jam Recovery Model). A jam at segment b stops all flow through b for duration $T_{\text{jam}}^{(b)} \sim \text{Weibull}(k_b, \lambda_{\text{jam}}^{(b)})$, calibrated from historical jam response times. During jam recovery:

- Upstream segments accumulate packages at their input rate
- Downstream zones lose induction volume: $V_{\text{gap}} = \text{PPH}_{\text{design}} \cdot T_{\text{jam}}^{(b)}$ (v1.4 §13, Layer 2 distortion)
- Jam-breaker employees are reassigned, triggering the v1.4 overhead reclassification effect

This formalizes the **connection between jam dynamics and the v1.4 jam-breaker distortion model**: the Layer 1 and Layer 2 distortions in v1.4 §13 are the observable iGate/SOR consequences of a jam event whose physical dynamics are modeled here.

5.4 Chute Capacity and Sort Destination Accumulation

Definition 5.6 (Chute Queue). Each sort chute $c \in \mathcal{C}$ has physical capacity K_c (packages) and receives packages at rate $\mu_c(t)$ determined by the routing distribution and belt flow. Packages are removed at rate $r_c(t)$ (loading to outbound trailer). The chute occupancy evolves as:

$$\frac{dQ_c}{dt} = \mu_c(t) - r_c(t), \quad Q_c \in [0, K_c]$$

When $Q_c = K_c$ (chute full), the belt must hold packages at that deflector — creating a **back-pressure wave** that propagates upstream, reducing effective belt throughput. Chute fullness is a leading indicator of sort completion and outbound trailer load progress.

6. Package Composition Layer

6.1 Package Size Distribution and Belt Capacity

Definition 6.1 (Package Size Distribution). Packages are characterized by their footprint area a_i (cm²) on the belt. The empirical distribution F_a across a sort can be approximated as a mixture of two log-normals:

$$F_a = \pi_{\text{small}} \cdot \text{LogN}(\mu_s, \sigma_s^2) + (1 - \pi_{\text{small}}) \cdot \text{LogN}(\mu_l, \sigma_l^2)$$

corresponding to small/soft-goods packages (envelopes, polybags: $\mu_s \approx 200 \text{ cm}^2$) and large/box packages ($\mu_l \approx 800 \text{ cm}^2$). The mixing weight π_{small} varies by day-of-week and retail season.

Definition 6.2 (Belt Density). Effective belt density (packages/meter) is:

$$\rho_b = \frac{\bar{a}}{w_b \cdot g_{\min}}$$

where $\bar{a} = \mathbb{E}[a_i]$ is mean package footprint, w_b is belt width, and $g_{\min}$ is minimum required separation gap. Higher $\bar{a}$ (larger packages) reduces ρ_b and lowers the effective throughput capacity — a direct link between package composition and belt physics.

6.2 Flowable vs. Non-Flowable Classification

Definition 6.3 (Flowability). A package is **flowable** if it can be inducted through the standard chute deflector at belt speed s_b without requiring manual assist. Let $p_{\text{flow}}(t)$ be the fraction of flowable packages in the current induction stream. Non-flowable packages require manual deflection — reducing effective sorter PPH during high non-flowable periods (the “cube surge” effect observed operationally).

Theorem 6.1 (Effective PPH under Flowability Mix). Under fraction p_{flow} of flowable packages and $1 - p_{\text{flow}}$ non-flowable:

$$\text{PPH}_{\text{eff}} = p_{\text{flow}} \cdot \text{PPH}_{\text{flow}} + (1 - p_{\text{flow}}) \cdot \text{PPH}_{\text{nf}}$$

where $\text{PPH}_{\text{flow}} > \text{PPH}_{\text{nf}}$ reflects the additional time per non-flowable package. In steady-state, if non-flowable fraction rises sharply (e.g., a bulk-goods feeder arrives mid-sort), the sort will register a transient PPH drop without any change in staffing or effort — a composition effect that cannot be identified from iGate data alone.

6.3 SLIC-to-Zone Load Distribution

Definition 6.4 (SLIC Load Vector). Each feeder’s manifest carries a SLIC vector $\mathbf{s}_i = (s_{iz})_{z \in \mathcal{Z}} \in \mathbb{R}_+^{|\mathcal{Z}|}$ where s_{iz} is the number of packages destined for zone z . The aggregate zone load at time t is:

$$\mathbf{L}(t) = \sum_{i: \tau_i \leq t} \text{fraction_processed}_i(t) \cdot \mathbf{s}_i$$

Load imbalance $\sigma_L(t) = \text{std}(\mathbf{L}(t))$ is a measure of zone unbalance. High σ_L implies that some zones are overloaded (chutes filling faster, staffing understressed elsewhere) while others are underloaded — motivating the borrow/loan optimizer in v1.4 §14.

Theorem 6.2 (Pre-Sort Zone Load Forecast). Given GPS-derived feeder ETA estimates $\hat{\tau}_i$ and ORION manifests $\mathbf{s}_i$, the expected zone load by sort end is forecastable before the first package is inducted:

$$\hat{\mathbf{L}}_{\text{final}} = \sum_{i=1}^N \mathbf{s}_i \cdot P(\tau_i < T)$$

This forecast enables **pre-sort staffing optimization**: set initial zone allocations proportional to $\hat{L}_{z,\text{final}}$ rather than waiting for iGate data to reveal imbalance reactively.

7. Human Factors Layer

7.1 Fatigue Degradation Curves

Physical labor productivity is not constant over a 4-hour sort. Empirical studies in manual materials handling (Garg et al., 1978; NIOSH, 2021) establish that sustained high-intensity sorting leads to measurable performance degradation.

Definition 7.1 (Fatigue Factor). The fatigue factor for sorter j at time t into the sort is:

$$F_j(t) = 1 - \frac{\kappa_F \cdot t^{\gamma_F}}{1 + \kappa_F \cdot t^{\gamma_F}}$$

with $\kappa_F > 0$ and $\gamma_F \in (0.5, 1.5]$ calibrated from physiological workload studies. $F_j(0) = 1$ (no fatigue), $F_j(t) \rightarrow (1 - \kappa_F)/(1 + \kappa_F)$ as $t \rightarrow \infty$ (saturation). The exponent γ_F controls the onset speed: low γ_F produces early-sort degradation, high γ_F produces late-sort cliff.

Definition 7.2 (Experience-Modulated PPH). The expected PPH for sorter j with experience e_j (sorts completed) at time t is:

$$\text{PPH}_j(t) = \text{PPH}_j^\infty \cdot (1 - \exp(-\theta e_j)) \cdot F_j(t) \cdot (1 - \kappa_T \Delta T_{\text{amb}})^+$$

where PPH_j^∞ is the asymptotic skill level (estimated via the v1.4 hierarchical Beta-Bernoulli model), θ is the learning rate (experience curve parameter), and ΔT_{amb} is the temperature excess from Definition 4.5.

7.2 Break Schedule Discontinuities

Definition 7.3 (Break Event). A scheduled break for sorter j at time $t_{\text{break}}^{(j)}$ of duration $\Delta_b^{(j)}$ induces a discontinuity in the staffing count $N(t)$ modeled in v1.4 as a jump process. In v2.0, breaks are **pre-scheduled events** in the state space:

$$N(t) = N_0 - \sum_j \mathbf{1} \left[t \in \left[t_{\text{break}}^{(j)}, t_{\text{break}}^{(j)} + \Delta_b^{(j)} \right] \right]$$

The break schedule is a known control input, enabling the Kalman filter to treat breaks as **known interventions** rather than unexplained staffing drops — improving prediction accuracy during the typically-degraded Wind-Down phase.

7.3 Sorter Heterogeneity and Assignment Optimization

Definition 7.4 (Skill-Weighted Staffing). Given per-sorter skill estimates $\hat{\theta}_j^{(z)}$ for belt family z (from the v1.4 hierarchical Bayesian model), the expected PPH for assignment $\sigma : j \mapsto z$ is:

$$\text{PPH}_{\text{assignment}}(\sigma) = \sum_j \text{PPH}_j^{(\sigma(j))} \cdot F_j(t)$$

The optimal assignment $\sigma^*(t)$ maximizes $\text{PPH}_{\text{assignment}}(\sigma)$ subject to zone minimum staffing constraints. This is a weighted bipartite matching problem solvable in $O(n^3)$ via the Hungarian algorithm, or approximately in $O(n^2)$ via greedy skill-ranking.

8. Unified State Space Model

8.1 Full State Vector

The complete v2.0 state vector at time t is:

$$\mathbf{X}(t) = \begin{bmatrix} \hat{\tau} & \text{(feeder ETA vector, } N_F\text{-dim)} \\ W(t) & \text{(weather state, categorical)} \\ \lambda_b(t) & \text{(per-belt arrival rates, } |\mathcal{B}|\text{-dim)} \\ \rho_b(t) & \text{(per-belt occupancy, } |\mathcal{B}|\text{-dim)} \\ \mathbf{Q}_c(t) & \text{(chute queues, } |\mathcal{C}|\text{-dim)} \\ p_{\text{flow}}(t) & \text{(flowable fraction)} \\ \mathbf{L}(t) & \text{(zone load vector, } |\mathcal{Z}|\text{-dim)} \\ \mathbf{F}(t) & \text{(sorter fatigue vector, } N(t)\text{-dim)} \\ \mathbf{x}_{\text{ops}}(t) & \text{(v1.4 ops state: PPH, SQS, fidelity, } N(t)) \end{bmatrix}$$

Total state dimension: $\approx 2N_F + 2|\mathcal{B}| + |\mathcal{C}| + 2|\mathcal{Z}| + N(t) + 6$. For Chelmsford parameters ($N_F \approx 20$ feeders, $|\mathcal{B}| \approx 30$ belt segments, $|\mathcal{C}| \approx 120$ chutes, $|\mathcal{Z}| \approx 8$ zones, $N(t) \approx 40$ sorters), the state dimension is approximately **300**.

8.2 Extended Kalman Filter

Because the system dynamics are nonlinear (M/G/1 queue congestion, fatigue curves, jam hazard), a standard linear Kalman filter is insufficient. The **Extended Kalman Filter (EKF)** linearizes the dynamics around the current state estimate.

Definition 8.1 (EKF Prediction Step). At each snapshot interval $\delta t = 15$ min, the state prediction is:

$$\begin{aligned} \hat{\mathbf{X}}(t + \delta t|t) &= \mathbf{f}(\hat{\mathbf{X}}(t|t), \mathbf{u}(t)) \\ \mathbf{P}(t + \delta t|t) &= \mathbf{F}_t \mathbf{P}(t|t) \mathbf{F}_t^\top + \mathbf{Q}_t \end{aligned}$$

where $\mathbf{f}(\cdot)$ is the nonlinear state transition (incorporating queue dynamics, fatigue evolution, weather Markov transitions), $\mathbf{u}(t)$ is the control input (belt speed, staffing assignment), $\mathbf{F}_t = \partial \mathbf{f} / \partial \mathbf{X} |_{\hat{\mathbf{X}}(t|t)}$ is the Jacobian, and $\mathbf{Q}_t$ is process noise (modeling unmodeled disturbances).

Definition 8.2 (EKF Update Step). When a new iGate/SOR snapshot $\mathbf{z}(t)$ arrives:

$$\begin{aligned}\mathbf{K}(t) &= \mathbf{P}(t|t-1)\mathbf{H}_t^\top (\mathbf{H}_t\mathbf{P}(t|t-1)\mathbf{H}_t^\top + \mathbf{R}(t))^{-1} \\ \hat{\mathbf{X}}(t|t) &= \hat{\mathbf{X}}(t|t-1) + \mathbf{K}(t) (\mathbf{z}(t) - \mathbf{h}(\hat{\mathbf{X}}(t|t-1))) \\ \mathbf{P}(t|t) &= (\mathbf{I} - \mathbf{K}(t)\mathbf{H}_t)\mathbf{P}(t|t-1)\end{aligned}$$

where $\mathbf{h}(\mathbf{X})$ maps the full state to the observable subspace (iGate scan counts, SOR hours, GPS positions), $\mathbf{H}_t = \partial\mathbf{h}/\partial\mathbf{X}|_{\hat{\mathbf{X}}(t|t-1)}$, and $\mathbf{R}(t)$ is measurement noise (GPS accuracy, iGate scan delay, SOR entry latency).

Theorem 8.1 (v1.4 as Marginalized v2.0). The v1.4 Kalman filter is obtained from the v2.0 EKF by:

1. Marginalizing out $\hat{\tau}, W(t), \lambda_b, \rho_b, \mathbf{Q}_c, p_{\text{flow}}, \mathbf{L}, \mathbf{F}$ (all unobserved layers)
2. Treating their contributions as increased process noise $\mathbf{Q}_t^{\text{v1.4}} = \mathbf{Q}_t^{\text{v2.0}}|_{\text{marginalized}}$

The v1.4 prediction errors (unexplained PPH variance) are precisely the variance attributable to the marginalized layers. As each layer becomes observable, $\mathbf{Q}_t$ decreases and prediction improves. This theorem provides a formal quantification of the **value of each new sensor layer** in terms of RMSE reduction.

9. Integrated Optimization Framework

9.1 Real-Time Hub Throughput Maximization

Definition 9.1 (Hub Throughput Objective). The primary operational objective is to maximize total packages sorted by sort close T , subject to sort quality constraints:

$$\max_{\mathbf{u}(\cdot)} \int_0^T \lambda_{\text{sort}}(t) dt \quad \text{subject to:}$$

$$\begin{aligned}\text{SQS}(T) &\geq \text{SQS}_{\min}, \quad \forall b : \rho_b(t) < 1, \quad \forall c : Q_c(t) \leq K_c \\ N_z(t) &\geq N_z^{\min}, \quad s_b(t) \in [s_{\min}, s_{\max}]\end{aligned}$$

where $\mathbf{u}(t) = (\mathbf{N}(t), \mathbf{s}(t), \mathbf{d}(t))$ is the control vector: zone staffing assignments $\mathbf{N}$, belt speed settings $\mathbf{s}$, and door open/close decisions $\mathbf{d}$.

9.2 Pre-Sort Staffing Optimization Under Arrival Uncertainty

Theorem 9.1 (Pre-Sort Optimal Staffing). Given the pre-sort GPS-based volume forecast $\hat{\mathbf{L}}_{\text{final}}$ (Theorem 6.2) and feeder ETA distribution, the optimal initial staffing allocation is:

$$N_z^*(0) = \arg \min_{N_z \geq N_z^{\min}} \mathbb{E} \left[\int_0^T (\lambda_z(t) - N_z(t) \cdot \text{PPH}_z(N_z(t)))^2 dt \right]$$

Under the linear-capacity approximation $\text{PPH}_z(N_z) \approx \overline{\text{PPH}}_z$ (constant per-sorter rate), this reduces to:

$$N_z^*(0) = \left\lceil \frac{\hat{L}_{z,\text{final}}}{T \cdot \overline{\text{PPH}}_z} \right\rceil$$

The uncertainty in $\hat{L}_{z,\text{final}}$ propagates as staffing uncertainty: $\text{Var}[N_z^*(0)] = \text{Var}[\hat{L}_{z,\text{final}}]/(T \cdot \overline{\text{PPH}}_z)^2$, quantifying the staffing risk attributable to GPS ETA uncertainty.

9.3 Belt Speed Control Policy

Theorem 9.2 (Optimal Belt Speed). For a single belt segment with induction rate $\lambda_b(t)$ and cost function penalizing both congestion and over-speed (missort/damage risk):

$$s_b^*(t) = \arg \min_{s \in [s_{\min}, s_{\max}]} [\mathcal{C}_{\text{cong}}(\rho_b(t, s)) + \mathcal{C}_{\text{missort}}(s)]$$

where $\mathcal{C}_{\text{cong}}(\rho) = c_1 \cdot \rho/(1 - \rho)$ (M/G/1 delay cost) and $\mathcal{C}_{\text{missort}}(s) = c_2 \cdot (s/s_{\max})^\eta$ (missort cost increasing superlinearly with speed). The unconstrained optimum is:

$$s_b^* = \left(\frac{c_1 \lambda_b \rho_b}{c_2 \eta / s_{\max}^\eta} \right)^{1/(\eta+1)}$$

projected onto $[s_{\min}, s_{\max}]$. This policy increases belt speed when induction rate is high (prioritizing throughput) and reduces it when packages are irregular or misrouting risk is elevated (prioritizing quality).

9.4 Jam-Conditioned Re-Optimization

Definition 9.2 (Reactive Re-Planning Trigger). A jam event at segment b at time t_{jam} triggers re-optimization of:

1. **Belt speed on adjacent segments** (reduce upstream rate to prevent cascade)
2. **Staffing re-assignment** (redeploy jam-breakers from low-urgency zones)
3. **Door management** (temporarily close high-rate doors feeding b)
4. **Sort completion forecast update** (push end-time estimate with confidence interval)

The re-optimization uses the current EKF state estimate $\hat{\mathbf{X}}(t_{\text{jam}}|t_{\text{jam}})$ as the new initial condition and runs forward simulation to produce updated recommendations within $\delta t_{\text{replan}} < 30$ seconds.

10. Digital Twin Architecture

10.1 What is a Hub Digital Twin?

A **digital twin** of the sort hub is a continuously-updated, high-fidelity simulation of the physical sort that runs in parallel with the live operation, ingesting real sensor data as it arrives and producing:

1. **Current state estimates** — what is happening right now at every belt, chute, and zone
2. **Predictive simulations** — what will happen in the next 15, 30, 60 minutes given current trajectory
3. **Counterfactual analysis** — what would have happened with different control decisions
4. **Prescriptive recommendations** — what should be done now to improve outcome

10.2 Data Ingestion Architecture

Definition 10.1 (Sensor Fusion Hub). The digital twin aggregates data from six stream types:

Stream	Update Rate	Protocol	Latency
iGate scans	Per scan (~100/min peak)	REST API push	<5 sec
SOR entries	Per entry (~1/5 min)	REST API push	<30 sec
GPS telematics	Per vehicle, 10 sec	MQTT stream	<15 sec
Belt speed sensors	Per segment, 1 sec	OPC-UA	<2 sec
Weather API	5-minute intervals	NOAA/NWS REST	<60 sec
Traffic API	1-minute intervals	HERE/Waze REST	<30 sec

All streams are timestamped at source and aligned to a common timeline via the EKF observation model (§8.2), which handles asynchronous observations naturally.

10.3 Predictive Simulation Engine

Definition 10.2 (Monte Carlo Forward Simulation). Given current EKF state estimate $\hat{\mathbf{X}}(t|t)$ and covariance $\mathbf{P}(t|t)$, the digital twin generates $M = 1000$ Monte Carlo trajectories by:

1. Sampling $\mathbf{X}_0^{(m)} \sim \mathcal{N}(\hat{\mathbf{X}}(t|t), \mathbf{P}(t|t))$
2. Propagating each sample forward via the nonlinear dynamics $\mathbf{f}(\cdot)$ with sampled process noise
3. Computing the distribution of any output quantity of interest (PPH at $t+30$, sort completion time, total volume sorted)

The 5th, 50th, and 95th percentile trajectories are displayed as the prediction band — replacing the point estimate currently shown in the v1.4 DOP Calculator with a **probabilistic forecast**.

10.4 Closed-Loop Control Interface

Definition 10.3 (Recommendation Engine). The digital twin’s closed-loop control module runs the integrated optimization (Section 9) at each snapshot and produces a **prioritized recommendation queue** for the coordinator:

```
[18:32] RECOMMENDATION: Redirect 3 sorters from Zone 4 → Zone 7
Reason: Feeder #12 (1,840 pcs, 82% Zone 7) ETA 18:45
Expected PPH gain: +12 PPH in Zone 7
Confidence: 87%
Time window: Act before 18:40

[18:45] ALERT: Belt B-7 density 0.89 (threshold 0.75)
Recommendation: Reduce induction at Door 14 by 30%
Jam probability in next 10 min: 34%
```

Recommendations are ranked by expected throughput impact and confidence, and time-windowed to give the coordinator actionable lead time.

11. Connection to v1.4

11.1 Formal Embedding

Theorem 11.1 (v1.4 as Restricted Case). Every theorem in the v1.4 framework (Theorems 3.1–14.1) remains valid in v2.0. Formally:

$$\mathcal{F}_{v1.4} = \mathcal{F}_{v2.0} \Big|_{\mathbf{x}_{\text{ext}}=\text{unobserved}, \mathbf{x}_{\text{env}}=\text{unobserved}, \mathbf{x}_{\text{infra}}=\text{unobserved}, \mathbf{x}_{\text{pkg}}=\text{unobserved}, \mathbf{x}_{\text{human}}=\text{unobserved}}$$

Proof sketch. v1.4 models only the projection $\mathbf{x}_{\text{ops}}$ of the full state $\mathbf{X}$. All v1.4 quantities (PPH, SQS, $N(t)$, FidelityScore, SQS components) are subcomponents of $\mathbf{x}_{\text{ops}}$. The v1.4 Kalman filter is the Kalman filter for the $\mathbf{x}_{\text{ops}}$ subsystem with inflated process noise absorbing the unobserved layers. The v1.4 Sort Quality Score, jam-breaker distortion corrections, and borrow/loan optimizer are policies defined on the $\mathbf{x}_{\text{ops}}$ subspace — all remain valid restrictions of their v2.0 counterparts. $\square$

11.2 Migration Path: v1.4 $\rightarrow$ v2.0

Each new sensor layer that becomes available incrementally reduces the EKF process noise and improves prediction accuracy. The migration path is:

Step	New Data Source	Expected RMSE Reduction	Complexity Addition
v1.5	SOR real-time push (vs. manual export)	~5–8%	Minimal
v1.6	Belt speed telemetry integration	~8–12%	Low
v1.7	ORION manifest feed (SLIC pre-sort)	~12–18%	Medium
v1.8	Feeder GPS ETA (internal UPS telematics)	~15–25%	Medium
v1.9	P-car GPS / ORION return estimates	~8–12%	Medium
v2.0	Weather + traffic + full EKF	~20–30% total over v1.4	High

Each step along this path is self-contained and independently deployable. v1.5 through v1.9 are the natural incremental extensions of the current validated baseline. v2.0 is the terminus.

12. Data Requirements

For v2.0 to be instantiated in production, the following data streams must exist and be accessible:

12.1 Required Data Sources

Data	Source	Access Mechanism	Status
iGate scan export	Internal iGate API	Current: manual export → v1.5: push API	Partially available
SOR hours	Internal SOR system	Current: manual export	Partially available
Feeder GPS positions	UPS Telematics / Coyote	UPS internal API or telematics vendor	Not currently integrated
P-car GPS / ORION return	UPS ORION system	UPS internal feed	Not currently integrated
Belt speed sensors	PLC / SCADA system	OPC-UA or existing SCADA interface	Infrastructure exists; API unknown
Induction rate counters	Induction station sensors	Same SCADA	Infrastructure exists; API unknown
Chute load sensors	Load cells (if installed)	SCADA	May not be installed at Chelmsford
Weather data	NOAA NWS / OpenWeatherMap	Public REST API	Easily accessible
Traffic data	HERE / Waze for Cities	Commercial API	Requires subscription
Package dimensions	Vision system (if installed)	Computer vision API or existing QA system	Unknown; may require capital investment
SLIC manifests	UPS Hub Management System	Internal feed	Unknown access level

12.2 Minimum Viable v2.0

A **partial v2.0** achieving the largest prediction improvement at lowest integration cost would require:

1. **Feeder GPS ETA** (largest single gain: ~15–25% RMSE reduction)
2. **NOAA weather API** (free, easily integrated)
3. **Belt speed telemetry** (infrastructure likely exists)
4. **ORION manifest pre-sort feed** (SLIC distribution known before sort start)

These four sources constitute the highest-leverage additions and could be integrated as v1.7–v1.8 milestones before reaching the full v2.0 EKF.

13. Limitations of the Idealized Model

13.1 Sensor Availability and Data Quality

The v2.0 framework assumes all sensor streams are available, reliable, and low-latency. In practice:

- GPS positions update every 10 seconds but may have gaps in dead zones (underground loading docks, thick warehouse walls)
- Belt speed sensors may not exist on all segments at Chelmsford; capital investment may be required
- Weather affects ETAs non-uniformly across the local road network; single-station weather may not capture hyperlocal road conditions
- ORION manifests are probabilistic (drivers may not follow route order); actual return package counts can deviate substantially from forecast

13.2 Model Identifiability

The full v2.0 state vector has 300 dimensions. The iGate/SOR observation vector at each snapshot has 50 dimensions. The system is **severely underidentified** from iGate/SOR alone; the EKF requires meaningful prior distributions on all latent states. In practice, this means:

- Belt density ρ_b must be initialized from physics (belt speed and induction rate, if available) rather than inferred from iGate data
- Fatigue factors $\mathbf{F}(t)$ require either wearable sensor data or strong empirical priors from physiological studies
- Package composition p_{flow} requires either vision systems or historical manifest-based estimates

13.3 Computational Tractability

The EKF Jacobian computation over a 300-dimensional state at 15-minute intervals is feasible ($O(300^2) \approx 90,000$ operations per update step). However, the Monte Carlo simulation engine (1,000 trajectories $\times$ 240 time steps $\times$ state dynamics) requires approximately 2.4×10^8 floating-point operations per forecast — on the order of seconds on modern hardware, acceptable for a 15-minute update cycle, but requiring dedicated compute (not a browser-based tracker).

13.4 Human Behavior Model Limitations

The fatigue and experience models are borrowed from general materials-handling literature. UPS-specific calibration would require:

- Controlled studies linking time-on-task to observed PPH degradation
- Per-sorter experience curves from longitudinal iGate history
- Temperature correlation studies in the Chelmsford facility specifically

These are feasible research projects but require data collection protocols not currently in place.

14. Future Research Directions

14.1 Reinforcement Learning for Real-Time Control

The integrated optimization (Section 9) is solved as a sequence of static problems at each snapshot. A more powerful approach is to formulate the full-sort control problem as a **Markov Decision Process** (MDP):

- **State space:** the v2.0 state vector $\mathbf{X}(t)$
- **Action space:** belt speed adjustments, staffing re-assignments, door open/close decisions
- **Reward:** sort-end throughput minus delay and quality penalties
- **Horizon:** 240 minutes, discretized at 15-minute intervals

Modern deep reinforcement learning (Proximal Policy Optimization, Soft Actor-Critic) could train a policy on simulated sorts generated by the digital twin. The trained policy would then operate in real-time, mapping observed state to control actions. This represents the full closure of the perception-action loop.

14.2 Multi-Facility Extension

The mathematical abstractions in v2.0 (marked point process, M/G/1 belt queue, fatigue curve, EKF) are facility-agnostic. The parameters (h_0 jam baseline, κ_F fatigue rate, α_w weather attenuation) are Chelmsford-specific. A **federated multi-facility model** would pool data across UPS hubs to improve parameter estimation while allowing facility-specific calibration — a hierarchical Bayesian structure at the facility level, analogous to the per-sorter Beta-Bernoulli model in v1.4.

14.3 Anomaly Detection and Root Cause Analysis

The digital twin provides a principled framework for **anomaly detection**: when observed $\mathbf{z}(t)$ deviates from EKF prediction by more than $k\sigma$, an anomaly is flagged. The EKF residual $\mathbf{z}(t) - \mathbf{h}(\hat{\mathbf{X}}(t|t-1))$ carries causal information: residuals in belt density suggest infrastructure issues, residuals in zone load suggest composition effects, residuals in fidelity suggest scanning compliance issues. Automatic root-cause attribution from the residual pattern is a natural extension.

14.4 Sort Simulation as Training Ground

The digital twin can generate synthetic sorts with controlled parameters (feeder delay, weather event, jam at specific belt, non-flowable surge). These synthetic sorts can serve as **training scenarios** for new coordinators — a formal extension of the LTC system from per-sorter routing training to per-coordinator operational decision training. This closes the three-timescale loop at the management level.

14.5 Jam-Breaker Identification from Belt Telemetry

Currently, jam detection relies on coordinator observation and manual coding. With belt speed sensors and induction rate counters, jam events can be detected automatically (belt speed drops to zero while induction rate remains positive). This would enable automatic application of the v1.4 jam-breaker distortion correction without relying on overhead labor code compliance — reducing the data fidelity requirement that currently limits SQS accuracy.

15. From Files to Equations: A Plain-Language Guide

This section requires no mathematical background. It describes, in plain language, what the software tools in this project actually do, why they were built, and how every decision Rafael makes on the sort floor connects — through those tools — to the mathematical model described in this paper.

15.1 What the Raw Files Actually Are

Every night at the UPS Chelmsford hub, two systems generate data exports:

iGate is the automated package scanning system. Every time a package passes a scanner on the belt, iGate records who scanned it (the employee), what was scanned (the barcode), and when it happened. At the end of the sort — or at any point during it — a supervisor can pull an export from iGate. This export is a spreadsheet (or CSV) that shows, for each employee, how many packages they scanned during a given time window. From this raw count, you can compute **PPH** (packages per hour): divide total packages scanned by total hours worked, and you have a measure of throughput. This is the heartbeat metric of the sort.

SOR (Staffing and Operations Record) is the manual system where supervisors and coordinators log who worked, in which area, for how long. If a sorter is borrowed from Zone 4 to help Zone 7, that transfer — if logged correctly — appears in SOR. If an employee works overtime, or clocks in late, SOR is where that is recorded. SOR data tells you how many people-hours were actually consumed in each area.

The limitation of both exports is that they are **snapshots in time**. You pull them at 8:30 PM and you get a picture of everything that happened from 6:00 PM to 8:30 PM. Pull them again at 10:00 PM and you get the full sort. If you pull them multiple times during the sort, you get a series of overlapping snapshots — and the **difference** between consecutive snapshots tells you what happened in that interval. This is the core data problem this project was built to solve.

15.2 What the HTML and JavaScript Files Do

The **Hub Operations Tracker** (the `tracker_v1.x.html` file) is a single web page you open in a browser. It has no server — all the logic runs in your browser using JavaScript. Its job is to take those raw iGate and SOR exports and turn them into an operational picture you can actually read during the sort, not just after it.

When you upload an iGate export into the tracker, the JavaScript code reads every row: who scanned, how many packages, over what time window. It then computes PPH for each employee, each area, and the entire hub. It color-codes the results — green means performing well, yellow means marginal, red means struggling — using the same thresholds that the hub’s own dashboard uses, so you are looking at the same interpretation your district managers would see.

When you upload a SOR export, the tracker reads hours by area and employee. It uses this to cross-check the iGate data: are the hours in SOR consistent with the scan records in iGate? If an employee has 300 scans in iGate but SOR shows them working in a different area, the tracker flags this as a fidelity issue. This is the **FidelityScore** in the mathematical model — the fraction of scans that are attributable to employees correctly logged in the right area.

When you upload files from **different times during the same sort** (a 7:45 PM export and a 9:30 PM export, for example), the tracker compares them. It can show you how PPH evolved over the sort, which areas peaked early and which held steady through wind-down, and where things started to slip. This is the **timeline reconstruction** — turning a series of flat spreadsheets into a living operational record.

The **DOP Calculator** tab in the tracker is a projection tool. You tell it: here is my current scan count, here is my current hour count, here is how much time is left. It computes whether you are on track to hit your target volume, and what PPH you need to sustain for the rest of the sort to get there. This is a simplified version of the Kalman filter prediction — it is doing forward projection from the current moment, though without the probabilistic uncertainty bands that the full mathematical model provides.

The **Hub Operations Container** (`hub_operations_v3.x.html`) is a wrapper around the tracker and the operations dashboard — it lets you switch between the live view and the historical view without leaving the browser.

15.3 What the Python Files Do

The **MasterTrendAnalysis extractor** (the Python scripts in the MasterTrendAnalysis folder) does something different from the tracker. Rather than analyzing a single sort in real time, it reads the archive of historical sort exports — 633 of them, spanning multiple years and three distinct operational eras — and structures them into a single dataset.

For each historical sort, the extractor reads the iGate and SOR exports and pulls out the key metrics: total volume, total hours, average PPH, area-level breakdown, date, day of week, season, era (which belt configuration was in use). It saves all of this as a structured JSON file — a clean, machine-readable record of every sort in the archive.

The `MasterTrendAnalysis/app/index.html` file then reads that JSON and lets you visualize trends: how does Chelmsford PPH on a Monday in February compare to a Thursday in November? How did performance change when the second SLS belt was added? The four prediction algorithms (KDE, DTW, weighted median, Kalman filter) run in the browser, each computing its own estimate of “how should today’s sort be going based on all past sorts that looked like today.”

Together, the Python extractor and the HTML prediction app are the **per-sort timescale** of the mathematical framework: they operate on historical data, across sorts, to give context and prediction for any new sort.

15.4 How Rafael’s Decisions Map to the Mathematical Model

Every decision made on the sort floor — every borrow, every loan, every door closure, every belt speed adjustment, every jam response — appears in the mathematical framework as a **control input** $\mathbf{u}(t)$.

Here is a plain-language translation of that mapping:

“I’m moving three people from Zone 4 to Zone 7 because Zone 7 is getting buried.”
In the model, this is an update to the staffing vector $N_z(t)$: $N_4(t)$ decreases by 3, $N_7(t)$ increases

by 3. The marginal productivity theorem (v1.4 §14) says this move is mathematically optimal if the marginal gain in Zone 7 exceeds the marginal loss in Zone 4 — which is exactly the intuitive judgment you are making when you recognize that Zone 7 is “more buried” than Zone 4 can afford to give up help.

“There’s a jam on B-line. I’m sending two jam-breakers.” This is the jam-breaker distortion model (v1.4 §13). The jam-breakers are physically pulling packages off the belt — real work, real throughput contribution. But because they are coded under an overhead labor code (not a scan-producing area code), iGate records their time as non-productive. The result: denominator (hours) stays high but numerator (scans attributed to them) goes down. The mathematical correction model formalizes this so that future analysis can strip out the artificial PPH suppression their presence caused.

“I’m closing Door 14 because that feeder is stacking up and I can’t induct fast enough.” This is the induction ceiling theorem (v2.0 §5.2). You are observing $\lambda_{\text{raw}}(t) > C_{\text{induct}}$ — more packages arriving than the belt can take in — and responding by reducing λ_{raw} at that door. The mathematical model tracks the resulting induction buffer B_{induct} and predicts when it will clear, which tells you when you can safely reopen the door without spiking belt density again.

“I’m holding off starting the sort for 10 minutes because the feeder from Boston is almost here.” This is GPS-based ETA prediction (v2.0 §3). You are implicitly reasoning about the volume distribution over the sort window: starting now with low inbound rate, then getting a 1,800-piece pulse at minute 10, is worse than waiting 10 minutes and having a more uniform inbound rate through the sort. The marked point process model formalizes this reasoning and can compute the expected throughput under both scenarios.

“PPH looks great right now but it’s 8:15 and we’ve barely touched the big Zone 3 feeder.” This is the phase-aware Markov chain (v1.4 §6.2) combined with volume forecast. You are recognizing that the current phase (Steady, favorable PPH) will give way to a late-arriving volume spike that changes the operational picture. The Kalman filter, when given the feeder ETA, adjusts its sort-end prediction accordingly — what looks like a strong sort at minute 135 may become a compressed wind-down race.

“I’m pushing belt speed up because we’re behind.” This is the belt speed control policy (v2.0 §9.3). Increasing belt speed raises throughput capacity (μ_b increases) but also raises missort risk (packages pass deflectors faster) and reduces sorter reaction time per package. The optimal policy trades these off explicitly. What you experience as an intuitive judgment — “we’re enough behind that it’s worth the risk” — is the model’s cost function crossing a threshold.

15.5 Why Every Intervention Creates a Ripple

The fundamental insight of the v2.0 framework — and the reason this model is complex — is that the hub is not a collection of independent systems. Every action propagates.

When you move three sorters from Zone 4 to Zone 7, Zone 4’s throughput drops. If Zone 4 was already at its scan-minimum, its chute may now fill more slowly — which sounds fine, but it means downstream trailers load later, which affects outbound sort schedule. Meanwhile Zone 7’s throughput rises, its chute fills faster, and if the loading team hasn’t been notified, you now have full chutes and a bottleneck at the trailer end.

When you close Door 14, the packages staged on the dock don't disappear. They accumulate. When you reopen the door, they hit the belt in a concentrated burst — a mini-spike that propagates as elevated belt density, increased jam probability, and a short-term PPH dip as sorters process the backlog.

When you send jam-breakers, the zones they came from lose staffing. Their PPH will show a temporary dip in the next iGate snapshot — which may look like a performance problem but is actually a consequence of your correct jam response. The jam-breaker distortion model (v1.4 §13) gives you the mathematical tool to separate “we underperformed” from “we responded to a physical event and the data makes it look like we underperformed.”

In the language of the model: the hub is a **coupled dynamical system**. Every control input $\mathbf{u}(t)$ changes the state $\mathbf{X}(t)$, which then evolves according to the belt queue dynamics, the staffing model, and the sorter channel — producing new observations at the next iGate snapshot. Your interventions are not corrections to an otherwise stable system. They are participation in the dynamics of the system — each one a perturbation that reshapes the trajectory.

The mathematical framework does not make this more complicated than it already is. It makes it *legible*. The equations give names and structure to what is already happening on the floor. The gap between the current tools (v1.4, the tracker, the extractor) and the idealized model (v2.0, the digital twin) is not a gap in understanding — it is a gap in sensor coverage. The understanding, formalized here, is already complete.

15.6 What This Means for the Tracker Going Forward

The tracker currently tells you what happened. The digital twin tells you what will happen and what you should do about it. The path from one to the other — laid out in the v1.5 through v1.9 migration steps in Section 11.2 — is a series of data integration milestones, each one giving the model more to work with and shrinking the uncertainty in its predictions.

Every time you pull an iGate export mid-sort and upload it to the tracker, you are performing a **Kalman filter update**: you are giving the model a new observation, and the model revises its picture of the current state. Currently, that update is manual — you download, upload, read the color-coded table. In the digital twin, that update happens automatically, every 15 minutes, across all layers simultaneously.

The HTML and JavaScript files are the proof-of-concept. The Python extractor is the data pipeline prototype. The mathematical framework is the blueprint. The digital twin is what they converge toward.

16. References

Arrivals and Logistics: - Dantzig, G. & Ramser, J. (1959). The truck dispatching problem. *Management Science*, 6(1), 80–91. - Laporte, G. (1992). The vehicle routing problem: An overview. *European Journal of Operational Research*, 59(3), 345–358. - Agustina, D., Lee, C.K.M., & Piplani, R. (2014). Vehicle scheduling and routing at a cross docking center. *EJOR*, 234(1), 136–150.

Queueing Theory: - Kleinrock, L. (1975). *Queueing Systems, Volume 1: Theory*. John Wiley

& Sons. - Gross, D., Shortle, J., Thompson, J., & Harris, C. (2008). *Fundamentals of Queueing Theory*. 4th ed. Wiley. - Whitt, W. (2002). *Stochastic-Process Limits*. Springer.

Traffic and Transportation: - Bureau of Public Roads (1964). *Traffic Assignment Manual*. U.S. Department of Commerce. - Lord, D. & Mannering, F. (2010). The statistical analysis of crash-frequency data. *Transportation Research Part A*, 44(5), 291–305. - Daganzo, C.F. (1994). The cell transmission model. *Transportation Research Part B*, 28(4), 269–287.

Weather and Operations: - Seppänen, O., Fisk, W., & Lei, Q. (2006). Effect of temperature on task performance in office environment. *Lawrence Berkeley National Laboratory*. - NIOSH (2021). *Ergonomic Guidelines for Manual Material Handling*. DHHS (NIOSH) Publication No. 2007-131. - Garg, A., Chaffin, D., & Herrin, G. (1978). Prediction of metabolic rates for manual materials handling jobs. *AIIE Transactions*, 10(4), 373–380.

State Estimation: - Kalman, R.E. (1960). A new approach to linear filtering and prediction problems. *ASME Journal of Basic Engineering*, 82(1), 35–45. - Simon, D. (2006). *Optimal State Estimation*. Wiley. - Durbin, J. & Koopman, S.J. (2012). *Time Series Analysis by State Space Methods*. 2nd ed. Oxford. - Julier, S. & Uhlmann, J. (1997). A new extension of the Kalman filter to nonlinear systems. *SPIE Proceedings*, 3068, 182–193. [Unscented KF — alternative to EKF for highly nonlinear systems]

Statistical Process Control: - Page, E.S. (1954). Continuous inspection schemes. *Biometrika*, 41(1/2), 100–115. - Hawkins, D.M. & Olwell, D.H. (2010). *Cumulative Sum Charts and Charting for Quality Improvement*. Springer.

Bayesian Methods: - Gelman, A., Carlin, J., Stern, H., Dunson, D., Vehtari, A., & Rubin, D. (2013). *Bayesian Data Analysis*. 3rd ed. CRC Press. - Wakefield, J. et al. (2023). Hierarchical Bayesian models for personnel skill estimation. *PLOS Computational Biology*.

Machine Learning and Optimization: - Sutton, R. & Barto, A. (2018). *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press. - Schulman, J. et al. (2017). Proximal Policy Optimization algorithms. *arXiv:1707.06347*. - Varian, H.R. (2010). *Intermediate Microeconomics: A Modern Approach*. 8th ed. Norton.

Previously cited (v1.3–v1.4): - Shannon, C.E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27, 379–423. - Pearl, J. (2009). *Causality*. 2nd ed. Cambridge. - Silverman, B.W. (1986). *Density Estimation for Statistics and Data Analysis*. Chapman & Hall. - Sheather, S. & Jones, M. (1991). A reliable data-based bandwidth selection method. *JRSS-B*, 53(3), 683–690. - Gray, R. & Neuhoff, D. (1998). Quantization. *IEEE Transactions on Information Theory*, 44(6), 2325–2383. - Sakoe, H. & Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE TASLP*, 26(1), 43–49. - Pinker, E.J. & Shumsky, R.A. (2000). The efficiency-quality tradeoff of cross-trained workers. *Manufacturing & Service Operations Management*, 2(1), 32–48.

Appendix A: Session Log

Session	Date	Framework Version	Focus
1–3	2025	v1.0–v1.1	Routing, channel model, monoid algebra, corrections
4	2026-05	v1.2	KDE, DTW, Kalman, MLR, CUSUM research

Session	Date	Framework Version	Focus
5	2026-05-12	v1.3	Academic restructure, 13 theorems, 35+ citations
6	2026-05-12	v1.4	SQS, Jam-Breaker distortion, Predictive staffing
7	2026-05-12	v2.0	Idealized digital twin: GPS, weather, belt physics, EKF

Appendix B: New Symbol Glossary (v2.0 Additions)

Symbol	Definition
Φ	Inbound freight marked point process
τ_i	Arrival time of vehicle i
$\mathbf{m}_i$	Mark of vehicle i : $(V_i, \mathbf{s}_i, \text{type}_i)$
V_i	Piece count for vehicle i
$\mathbf{s}_i$	SLIC load vector for vehicle i
$\hat{\tau}_i(t)$	GPS-based ETA estimate at time t
$\sigma_{\tau,i}$	ETA uncertainty (std dev, minutes)
$W(t)$	Weather state at time t
α_w	Weather speed attenuation factor
λ_{acc}	Local accident rate per unit road length
γ_w	Weather accident multiplier
$\lambda_b(t)$	Package arrival rate at belt segment b
$\mu_b(s_b)$	Belt service rate at speed s_b
$\rho_b(t)$	Belt occupancy (traffic intensity)
$h_b(t)$	Jam hazard rate at segment b
$Q_c(t)$	Chute queue length at chute c
K_c	Chute physical capacity
$p_{\text{flow}}(t)$	Flowable package fraction
$\mathbf{L}(t)$	Zone load vector
$F_j(t)$	Fatigue factor for sorter j
κ_F	Fatigue rate parameter
γ_F	Fatigue onset exponent
e_j	Sorter experience (sorts completed)
$\mathbf{X}(t)$	Full v2.0 state vector
$\mathbf{F}_t$	EKF Jacobian of dynamics
$\mathbf{H}_t$	EKF Jacobian of observation
$\mathbf{K}(t)$	Kalman gain matrix
$\mathbf{Q}_t$	Process noise covariance
$\mathbf{R}(t)$	Measurement noise covariance

All v1.4 symbols remain unchanged and carry over to v2.0.

Version: 2.0 — Idealized Digital Twin Framework

Author: Rafael Almeida, Employee 6068314

Facility: UPS Chelmsford Hub

Branch: Research vision — separate from v1.4 production baseline and v1.5 incremental path

Date: May 2026

The v1.4 framework captures what we can measure. The v2.0 framework describes what we would know if we could measure everything.